Scipy et les équations différentielles FIP --- 2026/2027

1 Un peu de Python pour commencer

Cette partie est indépendante du sujet du jour

1.1 Les erreurs en Python

Comprendre les erreurs

- Il est important de lire et comprendre les messages d'erreurs
- C'est la dernière ligne du message qui indique la nature de l'erreur.
- Python indique la ligne et l'endroit d'où vient l'erreur.

Sans exécuter le code, êtes vous capable de donner la nature de l'erreur et le message qui va apparaître ?

```
# Coquille longueur/longeur
largeur = 1
longeur = 2
surface = largeur*longueur
```

```
from math import sqrt
sqrt(-1)
```

```
# Espace en trop devant le print
def f(x):
    y = 2*x
     print(x)
```

```
# Compter les parenthèses ?
def f(x, y, z):
    return 13*(x + y*(sin(x+z))

b = 2
print(f(1, 2, 5))
```

```
# Nous avons déjà vu cette erreur...
import numpy as np

def valeur_absolue(x):
    if x>=0:
        return x
    else:
        return -x

print(valeur_absolue(np.linspace(1, 2))
```

Le try except

Lorsqu'il y a une erreur en Python, par défaut, l'interpréteur va arréter d'exécuter le code. Il est possible, en exécutant une fonction, d'empécher ce mécanisme et de gérer l'erreur. Cela se fait avec une structure `try ... except`.

Voici un exemple :

```
x = None

while x is None:
    input_text = input('Entrez un nombre')
    try :
        x = float(input_text)
    except :
        print("Je ne peux pas convertir l'entrée en nombre")
```

Créer sa propre erreur

Lorsque l'on crée une fonction, il peut être nécéssaire de créer une erreur lorsqu'il n'est pas possible de faire ce qui est demandé. Prenons un exemple : on lance une balle à la vitesse v_0 vers le haut, calculer le temps qu'il faut pour qu'elle atteigne le plafond situé à une hauteur h du point de départ.

Voici ce qu'il faut faire :

```
from math import sqrt
def temps_arrivee(h, v_0, g=9.81):
    Delta = v_0**2 - 2*g*h
    if Delta<0:
        raise Exception("La balle ne touche pas le plafond.")
    return (v_0 - sqrt(Delta))/g

print(temps_arrivee(h=1, v_0=10))
print(temps_arrivee(h=1, v_0=1))
```

- Au lieu de lever une exception, on décide d'utiliser un print de la façon suivante. Ceci est une très mauvaise idée et ne va pas résoudre le problème. Pourquoi seule la dernière ligne produit une erreur ?

```
def temps_arrivee(h, v_0, g=9.81):
    Delta = v_0**2 - 2*g*h
    if Delta<0:
        print("La balle ne touche pas le plafond")
    else:
        return (v_0 - sqrt(Delta))/g

t_0 = temps_arrivee(h=1, v_0=1)
print(f"Le temps d'arrivée en seconde est {t_0} s")
print(f"Le temps d'arrivée en miliseconde est {t_0*1000} ms")
```

Exercice

- Créer une fonction qui calcule l'aire d'un triangle en utilisant la formule d'Héron (voir sur wikipedia)
- Si l'argument de la racine carré est négatif, cela signifie que le triangle n'existe pas. Indiquer ceci par un message d'erreur explicite.

Remarque finale

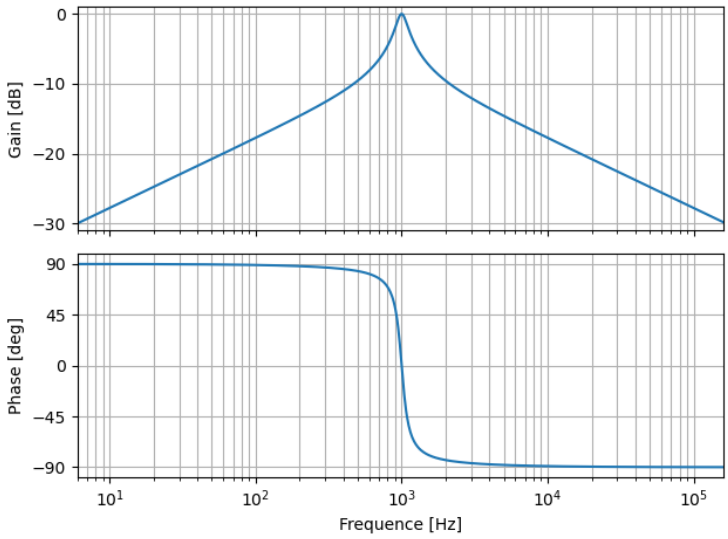
- Utiliser le try-except est gratuit s'l n'y a pas d'erreur. Cela ne sert à rien de faire le test explicite.
- Dans certains cas, même si on fait un texte préalable, il faudra de toutes les façons utiliser un try except : par exemple tester si un fichier existe avant de l'ouvrir ne sert à rien, car dans un système d'exploitation multitache, il est possible que le fichier soit supprimé entre le moment où on le crée et celui où il est ouvert.

1.2 Faire un graphique : diagramme de Bode

On condidère la fonction de transfert suivante :

$$H(j\omega) = \frac{A_0}{1 + jQ\left(\frac{\omega}{\omega_0} - \frac{\omega_0}{\omega}\right)}$$

Tracer le diagramme de Bode qui ressemblera à celui ci :



Paramètres : $\omega_0/(2\pi) = 1\text{kHz}$, $Q = 6$, $A_0 = 1$

On utilisera les fonctions suivantes :

- `subplots` pour faire deux graphs l'un sous l'autre (avec l'options `sharex=True`)
- `np.angle` pour extraire la phase d'un nombre complexe et `np.unwrap` pour qu'elle soit représentée de façon continue.
- `ticker.MultipleLocator` pour que la graduation se fasse à 45°.
- `grid` avec l'argument `which="both"` pour pouvoir voir les graduations intermédiaire de l'échelle log.

2 Scipy

SciPy est une librairie qui est complémentaire à NumPy et qui implémente des algorithmes pour résoudre des problèmes numériques.

2.1 Fonctions spéciales

SciPy implémente des fonctions qui ne sont pas dans NumPy comme les fonctions de Bessel, d'Airy, la fonction d'erreur, … Elles se trouvent dans le module `scipy.special`.

Tracer la fonction d'erreur (la primitive de $\frac{2}{\sqrt{\pi}}e^{-x^2}$) entre -5 et 5.

2.2 Fonctions spéciales

La fonciton `quad` de SciPy permet de calculer une intégrale numérique.

- Calculer numériquement $\int_0^1 cos(x^3/3)$
- Calculer numériquement $\int_0^\infty \frac{2}{\sqrt{\pi}}e^{-x^2}$ (on utilisera `np.inf`)

L'algorithme permet de trouver automatiquement les points où il faut évaluer la fonction. Il peut aussi calculer des intégrales avec une borne infinie. Lorsque l'on connaît la fonction, il ne faut donc surtout par utiliser un tableau de point !!

- Combien de point faut-il à scipy pour calculer $\int_0^\infty \frac{2}{\sqrt{\pi}}e^{-x^2}$? (pour cela on utilisera une variable globale ou un object que l'on modifiera pour enregistrer chaque appel de la fonction)

2.3 Optimisation

Nous avons déjà vu l'ajustement d'une courbe. Voici d'autres exemples d'optimisation

Zéro d'une fonction

- En utilisant la fonction `root_scalar`, calculer la plus petite solution non nulle de l'équation $\tan(x) = x$ (on pourra commencer par visualiser graphiquement l'équation puis afficher le résultat sur le graphique par un point)

Minimum

- En utilisant la fonction `minimize_scalar`, calculer le premier minium de la fonction sinus cardinal :

$$\text{sinc}(x) = \sin(x)/x$$

(et lire la documentation de `np.sinc` qui ne fait probablement pas ce que vous pensez)

Equation différentielle

La librairie `scipy.integrate` contient des fonctions pour résoudre les équations différentielles ordinaires, c'est à dire des équations de la forme :

$$\frac{dy}{dt} = f(t, y)$$

avec conditions initiales (on connait y à l'instant t_0). La variable y peut être un tableau numpy.

On utilise la fonction `solve_ivp` (remplace `ode` ou `odeint`):

```
solve_ivp(fun, t_span, y0, method='RK45', t_eval=None, ...)
```

Il existe plusieurs méthodes d'intégration (par défaut Runge-Kutta d'ordre 5(4) qui adapte la taille des pas)

La fonction `solve_ivp` renvoie un objet qui contient le résultat (res.y) mais aussi d'autres informations sur la convergence de l'algorithme.

Exemple : $\$\frac{dy}{dt} = -y\$$

- Créer la fonction `f(t, y)`. Cette fonction doit prendre l'argument `t` même si celui ci n'est pas utilisé

- Utiliser la `solve_ivp` pour résoudre l'équation différentielle. On prendra $y_0 = 1$ et `t_span=[0, 4]` et `t_eval=np.linspace(0, 4, 110)`. L'argument optionnel `t_eval` permet de lui demander des points intemédiaires pour pouvoir tracer l'évolution.
- Tracer la courbe
- Calculer la différence entre la courbe numérique et la courbe théorique. Jouer avec les paramètres `atol` et `rtol` pour réduire l'écart.
- Résoudre l'équation $y' = -y/\tau$ en définisant une fonction `f(t, y, tau)`

2.4 Equations différentielles d'ordre élevé

L'astuce consiste à augmenter la dimension de y en rajoutant des fonctions intermédiaires qui sont les dérivées de la fonction initiale.

Par exemple l'équation

$$\frac{d^2y}{dt^2} = \frac{f(y)}{m}$$

devient

$$\frac{d}{dt}\begin{pmatrix} y \\ y' \end{pmatrix} = \begin{pmatrix} y' \\ f(y)/m \end{pmatrix} = F(y, y')$$

Nous allons prendre l'exemple d'une masse sur un ressort. Il est conseillé de bien identifier les variables à l'intérieur de F et de ne pas hésiter à faire des appels à des fonctions dont les arguments ont plus de sens.

Voici ce que cela peut donner :

```
def force(x, k):
    return -k*x

def F(t, y, k, m):
    position, vitesse = y
    acceleration = force(position, k)/m
    return [vitesse, acceleration]
```

- Résoudre l'équation différentielle pour $k = 1$ et $m = 1$.

3 Le pendule

On considère l'équation : $\theta'' = -\sin\theta$

Pour résoudre cette équation, on définit le tableau $y(t) = (\theta(t), \theta'(t))$.

1. Écrire la fonction Python `f(t, y)` qui renvoie la dérivée de y
2. Résoudre et tracer le resultat de l'équation différentielle pour les conditions initiales : $\theta(0) = 3\pi/4$ et $\theta'(0) = 0$
3. Vérifier que l'énergie totale est conservée

4 Piège de Penning

À partir du sujet de 2021

En physique, les pièges à ions sont des dispositifs permettant de stocker des particules chargées pendant une longue durée, notamment dans le but de mesurer leurs propriétés avec précision.

Il n'est pas possible de réaliser un piège à ions uniquement avec un champ électrique statique : en effet, un tel piège nécéssiterait d'avoir un maximum ou minimum du potentiel électrique - ce qui est impossible car son Laplacien est nul.

Le piège de Penning est un dispositif permettant de stocker des particules chargées, grâce à la combinaison d'un champ magnétique uniforme et d'un champ électrique quadripolaire constant. Nous proposons d'en simuler le comportement.

On considère un ion béryllium Be^+ dont on notera q la charge. On note B l'amplitude du champ magnétique, uniforme, et dirigé selon z. Le champ électrostatique dérive du potentiel suivant :

$$V = \frac{Q}{2}\left(x^2 + y^2 - 2z^2\right)$$

Le champ électrique vaut donc :

$$\vec{E} = -Q\left(x\vec{u}_x + y\vec{u}_y - 2z\vec{u}_z\right)$$

La force est donnée par :

$$\vec{F} = q\left(\vec{E} + \vec{v} \wedge \vec{B}\right)$$

où le symbole $\wedge$ représente le produit vectoriel.

Valeurs numériques :

```
proton_mass = 1.67e-27
m = 9*proton_mass
q = e = 1.6e-19

# Valeurs par défaut
B = 2 # Tesla
Q = -300/1e-2**2 # 300V/cm^2
```

1. Ecrire une fonction python qui renvoie l'accélération $\vec{F}/m$ en fonction de $\vec{r}$ la position et $\vec{v}$ la vitesse représentée par des tableaux numpy. On pourra utiliser la fonction `np.cross` pour calculer la produit vectoriel
2. Ecrire une fonction qui à partir d'une condition initiale r_0, v_0 renvoie la position et la vitesse pour un ensemble de valeur (croissante) t_i

   ```
   def trajectoire(r_0, v_0, t_array, Q, B):
       ....
       return x, y, z, vx, vy, vz
   ```

3. Dans un champ magnétique uniforme (sans champ électrique), un particule à une trajectoire circulaire dans le plan orthogonal au champ. Vérifier ceci en calculant la tractoire d'une particule ayant initialement une vitesse de 100 m/s dans ce plan.
4. On considère maintenant une particule dans le champ magnétique et électrique. Paramètre initiaux : $\vec{r} = (0, 100\ \mu\text{m}, 0)$ et $\vec{v} = (500\ \text{m/s}, 100\ \text{m/s}, 1\ \text{m/s})$. On calculera pendant 5 μs en échantillonant sur au moins 2000 points. Tracer la trajectoire dans le plan x, y puis en 3D.

5 Nuage d'ions

On considère un nuage de N ions de masse m et charge q. On note $\vec{r_i}$ et $\vec{v_i}$ la position et la vitesse du ième ion. Les ions sont dans un piège électrostatique. De plus, ils interagissent entre eux par la force de Coulomb.

La force électrostatique dérive d'un potentiel électrostatique :

$$V(\vec{r}) = \frac{1}{2}k_x r_x^2 + \frac{1}{2}k_y r_y^2 + \frac{1}{2}k_z r_z^2$$

La force de Coulomb s'écrira sous la forme:

$$\vec{f_i}(\vec{r_i}, \vec{r_j}) = \kappa \frac{q^2}{|\vec{r_i} - \vec{r_j}|^3}(\vec{r_i} - \vec{r_j})$$

Pour simplifier et avoir une meilleure représentation graphique, on supprimera totalement la dimension z.

On utilisera des unités adimensionnées avec des constantes de l'ordre de 1. Par défaut on prendra : $m = 9$, $q = 1$, $k_x = 1$, $k_y = 1.3$, $\kappa = 1$.

L'état des N particules à un instant donné est représenté par 4 tableaux numpy de taille N: `r_x`, `r_y`, `v_x`, `v_y`. Toutes les fonctions seront écrites avec ces variables. On ne regroupera les 4 tableaux que dans la fonction `f(t, y)` qui sera utilisée par `solve_ivp`.

Le tableau `y` est défini en rassemblant les 4 tableaux numpy. On définit alors les fonctions `join` et `split` (voir ci-dessous)

Les paramètres seront des constantes globales.

1. Écrire la fonction `force_piege(r_x, r_y)` qui renvoie la force dérivant du potentiel (`f_x` et `f_y`).
2. Écrire la fonction `force_coulomb(r_x, r_y)` qui renvoie f_x et f_y, la force de Coulomb. On ne cherchera pas à éviter les boucles for dans cette fonction.
3. Écrire la fonction `f(t, y)` qui définit la dynamique du problème.
4. On condidère un nuage avec une distribution initiale donnée par une loi de Maxwell-Boltzmann (avec $k_B = 1$ et $T = 1$) pour les particules sans interaction. Calculer jusqu'au temps $T = 20$ l'évolution de la position des particules.
5. Vérifier que l'énergie totale est conservée. On utilisera les fonctions ci-dessous.
6. Faire une animation (voir exemple de code ci dessous)
7. On rajoute une force de dissipation, selon l'axe x :

   $$\vec{F}_{i,x} = -\alpha v_{i,x}$$

   Simuler l'experience en prenant $\alpha = 0.1$ et $N = 20$. Que se passe-t-il au temps long ?
8. Utiliser le decorateur `jit` de numba pour accélérer le code. Faire une comparaison de vitesse.
9. On considère un nuage contenant deux types de particules (masse m_1 et m_2) et de même charge $q = 1$. Seuls les particules de type 1 sont soumises à la force dissipative. Simuler l'expérience en prenant les mêmes paramètres qu'à la question 7. On prendra par exemple $m_1 = 9$, $N_1 = 20$ et $m_2 = 2$ et $N_2 = 5$

```
# Paramètres
m = 9; q = 1
k_x, k_y = 1, 1.3
kappa=1
k_B = 1
T = 1
```

Quelques fonctions pour vous aider :

- Fonction pour passer de 4 tableaux à un seul et réciproquement

```
def join(r_x, r_y, v_x, v_y):
    return np.concatenate((r_x, r_y, v_x, v_y))

def split(y):
    N = len(y)//4
    return y[:N], y[N:2*N], y[2*N:3*N], y[3*N:4*N]
```

- Calcul de l'énergie

```
def energie_cinetique(r_x, r_y, v_x, v_y):
    return np.sum(.5*m*v_x**2 + .5*m*v_y**2)

def energie_piege(r_x, r_y):
    return np.sum(k_x*r_x**2/2 + k_y*r_y**2/2)

def energie_coulomb(r_x, r_y):
    N = len(r_x)
    total = 0
    for i in range(N-1):
        for j in range(i+1, N):
            d2 = (r_x[i]-r_x[j])**2 + (r_y[i]-r_y[j])**2
            total += kappa*q**2/np.sqrt(d2)
    return total

def energie_totale(r_x, r_y, v_x, v_y):
    return (energie_coulomb(r_x, r_y) +
            energie_piege(r_x, r_y) +
            energie_cinetique(r_x, r_y, v_x, v_y))
```

```
from IPython.display import HTML
from matplotlib.animation import FuncAnimation

fig, ax = plt.subplots()
l, = ax.plot(split(res.y[:,0])[0], split(res.y[:,0])[1], 'o')
ax.set_xlim(-10, 10)
ax.set_ylim(-10, 10)

def animate(i):
    r_x, r_y, v_x, v_y = split(res.y[:,i])
    l.set_data(r_x, r_y)

ani = FuncAnimation(fig, animate, frames=len(res.t))


HTML(ani.to_jshtml())
```